Embedding Cover-Free Families (CFFs): Implementation and Applications in Cryptography

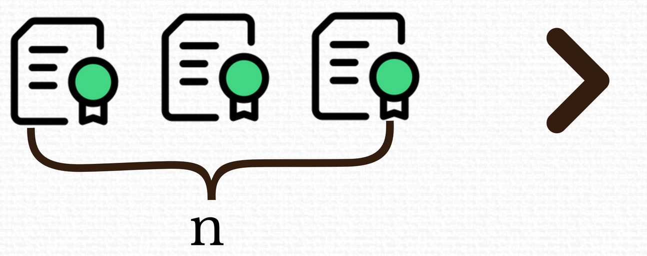
Luiz Roberto Scolari

joint work with Thaís Bardini Idalino

Introduction

What is a Digital Signature?

- Cryptographic proof of integrity and authenticity.
- Non-repudiation guarantee.



n documents
n signatures
n verifications?

Naive solution

- One verification per document: total of *n* verifications and *n* stored signatures.

Better solution

- Aggregate all signatures into one → verify the aggregate, not individual signatures.

Even better solution

- Uses *d*-Cover-Free Families (*d*-CFFs).
- Enables partial aggregation with fault detection.

$$M = \begin{array}{c} a_1 \\ a_2 \\ a_3 \\ a_4 \end{array} \begin{bmatrix} 1 & 1 & 1 & \emptyset & \emptyset & \emptyset \\ 1 & \emptyset & \emptyset & 1 & 1 & \emptyset \\ \emptyset & 1 & \emptyset & 1 & \emptyset & 1 \\ \emptyset & \emptyset & 1 & \emptyset & 1 & 1 \end{bmatrix}$$

***d*-Cover-Free Families (*d*-CFFs)**

Definition: a $t \times n$ binary matrix is a *d*-CFF(t, n) if any set of $d+1$ columns contains a permutation submatrix of order $d+1$.

- No column is "covered" by the union of any *d* others.
- For given *n* and *d*, minimize *t*.
- **Application**: Combinatorial Group Testing - efficiently identify up to *d* invalid elements out of *n* total elements.

Theorem (**E, F, F, 1985** [**1**]) Let *q* be a prime power, $k \geq 1$. Then, there exists a *d*-CFF(q^2, q^{k+1}), with $d \leq \frac{q-1}{k}$.

Polys over $\mathbb{F}_q$ of degree up to *k*

$(\mathbb{F}_q, \mathbb{F}_q)$

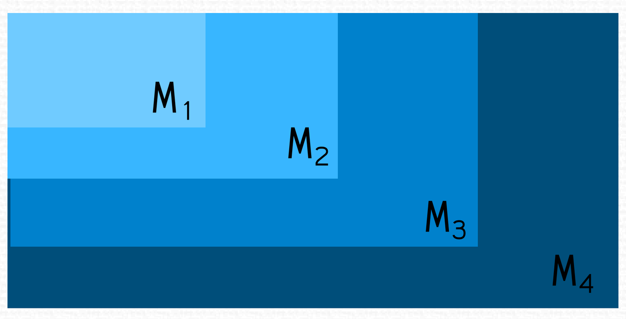
	0	1	2	x	x+1	x+2	2x	2x+1	2x+2
(0,0)	1	0	0	1	0	0	1	0	0
(0,1)	0	1	0	0	1	0	0	1	0
(0,2)	0	0	1	0	0	1	0	0	1
(1,0)	1	0	0	0	0	1	0	1	0
(1,1)	0	1	0	1	0	0	0	0	1
(1,2)	0	0	1	0	1	0	1	0	0
(2,0)	1	0	0	0	1	0	0	0	1
(2,1)	0	1	0	0	0	1	1	0	0
(2,2)	0	0	1	1	0	0	0	1	0

$q = \mathbb{F}_3$
$k = 1$
$d = 2$
2-CFF(9,9)

Embedding CFFs

Problem: Are CFFs limited to static problems?

$$M_{i+1} = \begin{pmatrix} M_i & X \\ Z & Y \end{pmatrix}$$



- Infinite sequence of CFFs.
- Incremental construction: M_{i+1} embeds M_i as a submatrix.
- Increasing both *n* and *t* → increase *d*.
- There are also some other types of families, such as monotone and nested families.

Theorem (**I, M, 2019** [**2**]) Let *q* be a prime power, $q \geq d_0 k_0 + 1$. Let $k_i \leq k_{i+1}$, $d_i \leq d_{i+1}$, $q^{2^i} \geq d_i k_i + 1$, for all $i \geq 0$. Then, the sequence $\{M_{q^{2^i}, k_i, d_i}\}_{i \geq 0}$ is an embedding family of CFFs.

- Use **extension fields** to generate embedding CFFs.
- The extension field ensures that smaller matrices are embedded within larger ones.

	0	1	2	x	x+1	x+2	2x	2x+1	2x+2	. . .	(2α+2)x²+ (2α+2)x+ 2α+2
(0,0)	1	0	0	1	0	0	1	0	0	. . .	
(0,1)	0	1	0	0	1	0	0	1	0		
(0,2)	0	0	1	0	0	1	0	0	1		
(1,0)	1	0	0	0	0	1	0	1	0		
(1,1)	0	1	0	1	0	0	0	0	1		
(1,2)	0	0	1	0	1	0	1	0	0		
(2,0)	1	0	0	0	1	0	0	0	1		
(2,1)	0	1	0	0	0	1	1	0	0		
(2,2)	0	0	1	1	0	0	0	1	0		
$(\mathbb{F}_3, \mathbb{F}_9 \setminus \mathbb{F}_3)$	. . .									. . .	
$(\mathbb{F}_9 \setminus \mathbb{F}_3, \mathbb{F}_9)$											
	0	0	0	1	0	0	0	0	0	. . .	

$\mathbb{F}_3 \rightarrow \mathbb{F}_9$
$k = 1 \rightarrow k = 2$
$d = 2 \rightarrow d = 4$
4-CFF(81,729)

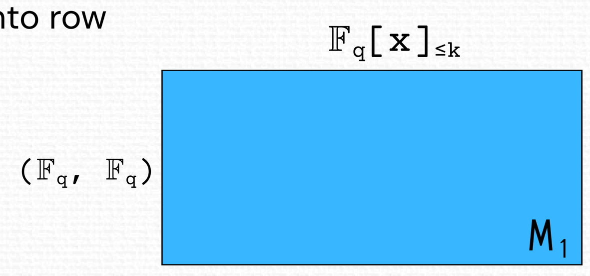
Implementation

Generate initial CFF

- Input: $\mathbb{F}_q$, *k*
- Output: d-CFF(t, n)
  - M ← []
  - for each (x, y) in $\mathbb{F}_q \times \mathbb{F}_q$:
    - row ← []
    - for each p in Polys
      - if p(x) = y:
        - insert 1 into row
      - else:
        - insert 0 into row
    - insert row into M

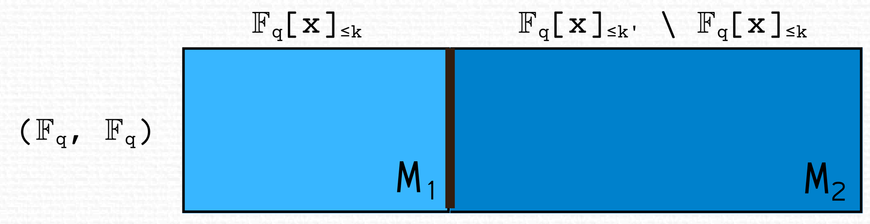
Naive evaluation

- $q^2 \times q^{k+1}$ evaluations



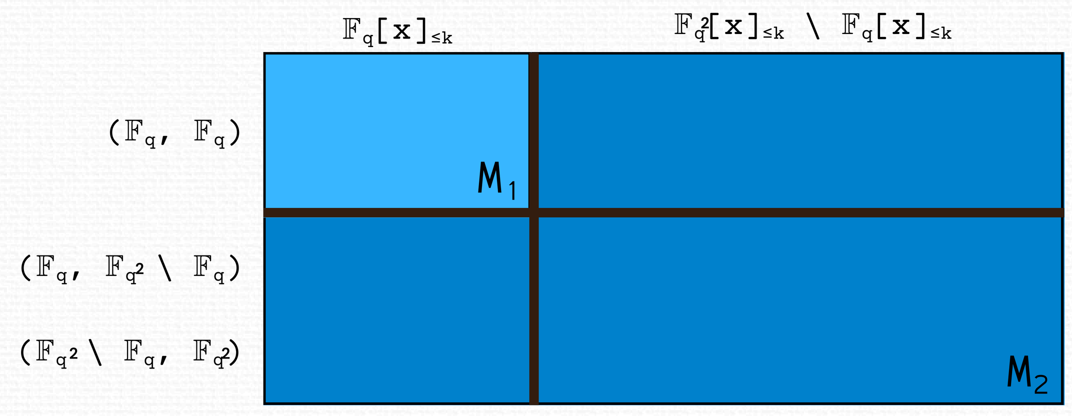
Increase the degree *k*

- Input: $\mathbb{F}_q$, *k* , *k'*
- Output: d'-CFF(t, n'), subject to $q \geq d'k' + 1$



Extend the finite field

- Input: $\mathbb{F}_q$, $\mathbb{F}_{q^2}$, *k*
- Output: d'-CFF(t', n'), subject to $q^2 \geq d'k + 1$



Optimization

Map element → row index

- Each element x of the finite field is mapped to its index in a precomputed list (ranking).
- Enables constant-time lookup of x's evaluation row: E[index[x]].

Precompute evaluations

- Build the evaluation matrix E.
  - Row *i*, column *j* stores $p_j(e_i)$

Better evaluation

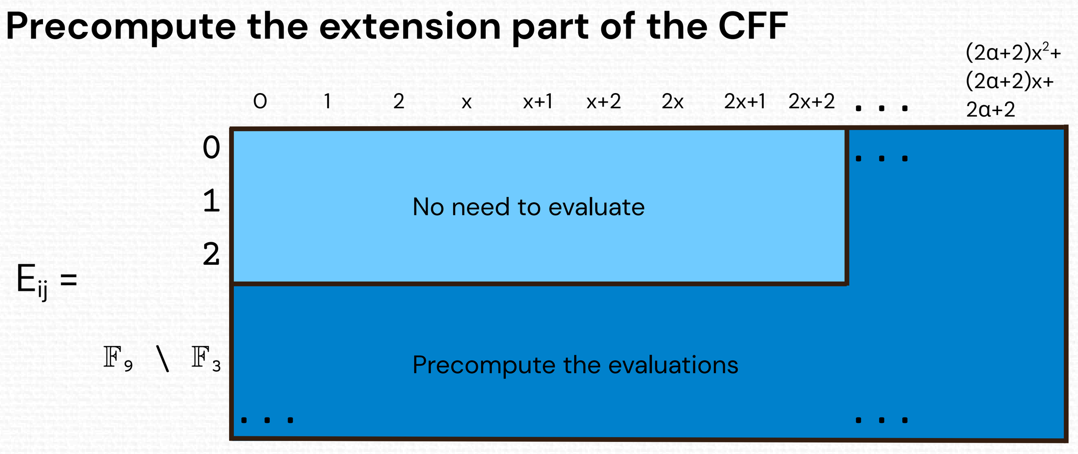
- $q \times q^{k+1}$ evaluations

$$E_{ij} = \begin{array}{c} \\ 0 \\ 1 \\ 2 \end{array} \begin{array}{ccccccccc} 0 & 1 & 2 & x & x+1 & x+2 & 2x & 2x+1 & 2x+2 \\ \hline 0 & 1 & 2 & 0 & 1 & 2 & 0 & 1 & 2 \\ 0 & 1 & 2 & 1 & 2 & 0 & 2 & 0 & 1 \\ 0 & 1 & 2 & 2 & 0 & 1 & 1 & 2 & 0 \end{array}$$

Build the CFF

- Input: $\mathbb{F}_q$, *k*
- Output: d-CFF(t, n)
  - *n* = number of polynomials
  - M ← []
  - For each (x, y) in $\mathbb{F}_q \times \mathbb{F}_q$:
    - *i* ← index[x]
    - row ← []
    - For *j* ← 0 to n - 1:
      - if E[i][j] = y:
        - insert 1 into row
      - else:
        - insert 0 into row
    - insert row into M

Precompute the extension part of the CFF



Results

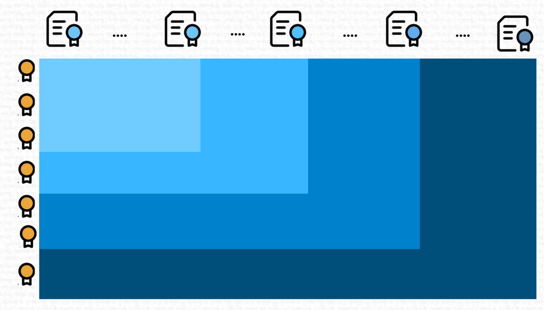
q	k	d	t	n	time(s) naive	time(s)
2	1	1	4	4	1,07	1,18
4	1	3	16	16	1,01	1,06
4	2	1	16	64	0,13	0,02
4	3	1	16	256	0,47	0,07
3	1	2	9	9	1,27	1,66
3	2	1	9	27	0,95	0,94
9	2	4	81	729	8,70	1,16
9	3	2	81	6.561	59,9	1,58
9	4	2	81	59.049	537,03	14,20
9	5	1	81	531.441	4.876,98	125,27
9	6	1	81	4.782.969	44.602,64	1098,38
5	1	4	25	25	1,33	1,25
5	2	2	25	125	0,27	0,04
25	2	12	625	15.625	1.099,40	4,82
25	3	8	625	390.625	26.154,60	94,07

Hardware Configuration
Processor: Intel Core i5-7400 @ 3.00GHz (4 cores)
Memory: 8 GB RAM

Applications

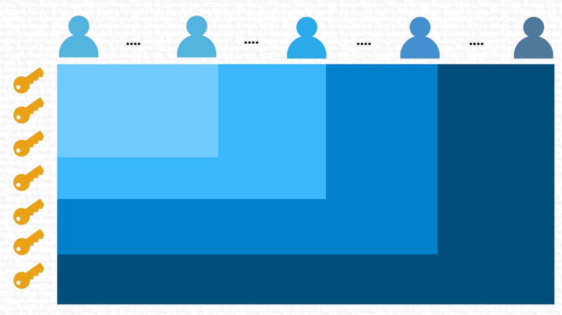
Digital signatures: fault-tolerance for digital signature applications.

- **Columns**: digital signatures/documents
- **Rows**: aggregated signatures
- **d**: upper-bound on the number of invalid signatures



Key distribution: tolerance to malicious users in a broadcast application.

- **Columns**: users
- **Rows**: keys
- **d**: upper-bound on the number of malicious users



Conclusion

- CFFs have several cryptographic applications, with much still to explore and study.
- Polynomial CFFs strike a balance among *n, t* and *d.*
  - focus on increasing only *n.*
  - focus on increasing both *n* and *d.*
- Generating large CFFs is slow — precomputing and storing CFFs is a practical workaround.

Future work

- Grow *q* further than p^2.
- Construct variations of growth.
  - monotone families and nested families.
  - smoother growth of *n* for embedding CFFs.
- Apply dynamic CFFs to concrete cryptographic problems.
- Use parallelism to construct CFFs.

Bibliography

[1] P. Erdős, P. Frankl and Z. Füredi, "Families of finite sets in which no set is covered by the union of r others," Israel Journal of Mathematics, vol. 51, no. 1/2, pp. 79-89, 1985.

[2] T. B. Idalino and L. Moura, "Embedding cover-free families and cryptographical applications," Advances in Mathematics of Communications, vol. 13, no. 4, pp. 629-643, 2019.

[3] T. B. Idalino and L. Moura, A survey of cover-free families: Constructions, applications, and generalizations, in New Advances in Designs, Codes and Cryptography, edited by C. J. Colbourn and J. H. Dinitz, Springer Nature Switzerland, Cham, 2024, pp.~195--239.

CAPES
CNPq Conselho Nacional de Desenvolvimento Científico e Tecnológico
São Carlos 2025 fq16
FAPESP
ELSEVIER